



M N I M E

MULTIMODAL NEURAL INTERFACE MACHINE EXTENSION
nigh.mh

Full Specification Sheet & User Manual

v5.0

100%
Offline

4.0x
Retina

60 FPS
UI

1.5B
NLP Params

TABLE OF CONTENTS

1	Product Overview
2	Technical Specifications
3	Architecture & Module Reference
4	Feature Reference
4.1	Combine PDF
4.2	JPG to PDF
4.3	TXT to PDF
4.4	PDF to JPG
4.5	Split PDF
4.6	Compress PDF
4.7	PDF to DOCX
4.8	Bookmark
4.9	Edit Suite
4.10	NLP Chat & RAG Engine
4.11	Reference Engine
5	NLP Engine — MNIME-Core
6	Semantic Search Engine
7	User Interface Guide
7.1	Tabs Bar
7.2	Drop Zone
7.3	Gallery Carousel
7.4	Action Bar
7.5	Document Reader
7.6	Edit UI
7.7	NLP Floating Chat
8	Installation & Setup
8.1	Standalone Installer
8.2	Run from Source
8.3	Build Pipeline
9	Keyboard Shortcuts & Tips
10	Performance Notes
11	Dependency Stack
12	Error Handling & Validation
13	Changelog
14	License & Attribution

1 PRODUCT OVERVIEW

MNIME — **Multimodal Neural Interface Machine Extension** — is a modern, private, and ultra-fast desktop document-processing application engineered with Python and PyQt6. It runs **100% locally and offline** on the host machine with zero external network uploads, giving users full conversational interaction over their documents via a bundled, fine-tuned local NLP model.

Attribute	Value
Full Name	Multimodal Neural Interface Machine Extension
Version	v5.0
Platform	Windows 10 / 11 (64-bit)
Language	Python 3.12+
UI Framework	PyQt6 6.11
License	Open Source (see Section 14)
Author	kyledeanml / KyleDeanAI
Repository	https://github.com/kyledeanml/MNIME
NLP Model	KyleDeanAI/MNIME-Core-V5-Q4_K_M (HuggingFace)
Network	Fully Offline — Zero external API calls

Design Philosophy

MNIME is purpose-built around three pillars: **privacy** (nothing ever leaves the machine), **performance** (C-accelerated document engines, background workers, O(1) UI operations), and **aesthetics** (a free-floating dark metallic interface with physics-based animations and a mathematically precise 5D Penteract projection logo). The application avoids all web service dependencies, making it suitable for air-gapped, regulated, or sensitive document environments.

2 TECHNICAL SPECIFICATIONS

Specification	Detail
Minimum OS	Windows 10 64-bit (Build 1903+)
Recommended OS	Windows 11 64-bit
Python Version	3.12 or higher
CPU	Any modern x86-64 processor (multi-core recommended)
RAM — Lightweight Mode	~1.0 GB (NLP disabled, base runtime)
RAM — NLP Active	~1.8–1.9 GB (model weights loaded in VRAM/RAM)
GPU (Optional)	NVIDIA CUDA-capable GPU for NLP layer offloading
GPU VRAM (Optional)	4 GB+ recommended for full GGUF layer offload
Validation / Dev Bench	AMD Ryzen 9 7950X, 64 GB DDR5 6000 MHz, RTX 3060 Ventus 12 GB
Disk Space	~2 GB (application + bundled model)
Display	1280×720 minimum; 1920×1080+ recommended
File Queue Limit	Up to 5,000 files per session
PDF Rendering DPI	4.0x Retina (288 DPI equivalent)
NLP Context Window	4,096 tokens
NLP Threads	8 CPU threads (llama.cpp)
NLP GPU Layers	Auto-detect (n_gpu_layers = -1)
Embedding Model	BAAI/bge-small-en-v1.5 (FAISS-backed)
Vector Index	FAISS (CPU), chunk size 1,500 / overlap 200
Image Conversion DPI	200 DPI default (PDF → JPG)
UI Framerate Target	60 FPS (QThread non-blocking)
Installer Format	Custom PyQt6 animated installer
Install Location	%LOCALAPPDATA%\Programs\MNIME

3 ARCHITECTURE & MODULE REFERENCE

MNIME is structured into two top-level packages — **core/** (backend processing engine) and **ui/** (PyQt6 frontend) — plus a root-level entry point. All heavy operations run in **QThread** workers to keep the UI at 60 FPS.

Module	Role	Key Dependencies
MNIME.py	Entry point. Initializes Qt app, sets AppUserModelID, applies global stylesheet, instantiates MainWindow.	PyQt6, core.app_icon
core/pdf_engine.py	All document transformations: merge, split, JPG↔PDF, TXT→PDF, compress, DOCX export, bookmark.	pymupdf, pypdf, pdf2docx, Pillow, concurrent.futures
core/nlp_engine.py	Singleton NLP engine. Loads/unloads GGUF model, generates chat responses, smart bookmarks, smart filenames.	llama-cpp-python
core/search_engine.py	FAISS-backed semantic search. Indexes PDFs/text files, performs similarity search, runs Windows OCR on image-only pages.	langchain, faiss-cpu, sentence-transformers, winsdk
core/file_item.py	Data model for a queued file. Stores path, extension, metadata, thumbnail pixmap.	pymupdf, Pillow
core/fusion_engine.py	Neural Assimilation Engine. Background weight-fusion system for dynamic model merging & parameter absorption.	numpy, torch, llama-cpp-python
core/worker.py	Generic QThread worker with progress_callback signal. Wraps all engine calls.	PyQt6.QtCore
core/app_icon.py	Windows AppUserModelID registration, ICO generation, taskbar/desktop shortcut creation.	winreg, ctypes
ui/main_window.py	Central coordinator. Frameless dark metallic window, mode routing, drag-and-drop handler, VFX trigger.	PyQt6, all ui/ modules
ui/tabs_bar.py	Icon-only mode switcher. Hosts NLP checkbox master toggle and Reload NLP button.	PyQt6
ui/carousel_view.py	Horizontal reorderable file card carousel with drop zone. O(1) index-based drag reorder.	PyQt6
ui/file_card.py	Individual card widget: thumbnail, filename, status badge, progress bar, remove (X) button.	PyQt6
ui/file_dialog.py	Custom dark-mode file explorer dialog replacing OS popup. Directory tree + file list.	PyQt6
ui/action_bar.py	Primary action button + animated progress bar.	PyQt6
ui/nlp_view.py	Conversational NLP chat interface. Sends queries to NLP Engine, displays streamed responses.	PyQt6
ui/output_view.py	Processing log / status view with timestamped entries.	PyQt6
ui/reader_dialog.py	Frameless independent document reader at 4.0x DPI. Ctrl+Scroll zoom, auto-fit.	PyQt6, pymupdf
ui/pdf_editor.py	PDF crop/rotate editor UI.	PyQt6, pymupdf
ui/image_editor.py	Image crop/rotate editor UI.	PyQt6, Pillow
ui/document_viewer.py	Embedded document viewer used in Reference mode.	PyQt6, pymupdf
ui/merge_particles.py	Physics-based particle simulation (file vortex) displayed during processing.	PyQt6
ui/minimize_animation.py	Custom minimize animation for frameless window.	PyQt6
ui/cursor_fx.py	Custom cursor effect applied globally.	PyQt6
ui/icons.py	Resolution-independent SVG icon registry. Returns QIcon at any DPI.	PyQt6
custom_installer.py	Standalone animated PyQt6 installer with flying-file progress bar.	PyQt6, PyInstaller
MNIME.spec	PyInstaller spec for main application bundle.	PyInstaller

4 FEATURE REFERENCE

4.1 Combine PDF

Property	Value
Inputs	PDF files, JPG/PNG/BMP/WEBP images, TXT files (mixed)
Output	Single merged PDF document
Max Files	5,000 files per session
Engine	PyMuPDF (primary) → pypdf + Pillow (fallback)
Algorithm	Sequential insert_pdf() with garbage=3, deflate=True compaction
Performance	C-level PyMuPDF routines, up to 50x faster than pure-Python

4.2 JPG to PDF

Property	Value
Inputs	JPG, JPEG, PNG, WEBP, BMP image files
Output	Single unified PDF document
Engine	PyMuPDF sequential convert_to_pdf() → Pillow fallback
Parallelism	Sequential (MuPDF is not thread-safe); runs off the UI thread
Save	garbage=3, deflate=True optimization

4.3 TXT to PDF

Property	Value
Inputs	.txt plain text files
Output	Searchable native vector PDF
Engine	PyMuPDF insert_textbox() (Helvetica, 12pt, 50pt margins)
Notes	Can be combined with PDFs and images in Combine mode

4.4 PDF to JPG

Property	Value
Inputs	Single PDF document
Output	One JPG image per page
DPI	200 DPI default (zoom = DPI / 72)
Engine	PyMuPDF get_pixmap() sequential rendering
Parallelism	Sequential (MuPDF is not thread-safe); runs off the UI thread
Naming	<basename>_page_<NNN>.jpg

4.5 Split PDF

Property	Value
Inputs	Single PDF document
Output	One PDF file per page, saved to output directory
NLP Smart Naming	If NLP is loaded, each page's text is sent to MNIME-Core to generate a concise, context-aware filename (under 40 chars, underscored). Page number appended for uniqueness.
Fallback Naming	<basename>_page_<NNN>.pdf
Parallelism	Sequential page loop (MuPDF is not thread-safe); NLP lock retained

4.6 Compress PDF

Property	Value
Inputs	Single PDF document
Output	Compressed PDF (overwrite-safe output path)
Engine	PyMuPDF save(garbage=4, deflate=True, clean=True) → pypdf fallback
Method	Stream deflation, xref compaction, duplicate object elimination

4.7 PDF to DOCX

Property	Value
Inputs	Single PDF document
Output	.docx Microsoft Word document
Engine	pdf2docx (full layout fidelity) → pypdf + python-docx (text fallback)
Fidelity	Preserves text layout, columns, and formatting via pdf2docx

4.8 Bookmark

Property	Value
Inputs	Single PDF document
Output	Bookmarked PDF with structured table of contents
Heuristics	Font-size analysis across sampled pages to detect body text baseline; headings identified by size $\geq 1.15\times$ baseline OR bold $\geq 1.05\times$. Explicit chapter/section regex matched first.
NLP Mode	If NLP is loaded, each detected heading is enriched via MNIME-Core into a verbose, context-aware bookmark title (≤ 60 chars)
Fallback	If no headings detected, all pages bookmarked using largest text found

4.9 Edit Suite

Property	Value
Scope	Images and PDF documents
Operations	Crop (drag selection), rotate (90° CW/CCW), apply to all pages or single page
Rendering	4.0x Retina pixmap rendering for razor-sharp editing canvas
Windows	Independent frameless dark metallic editor windows

4.10 NLP Chat & RAG

Property	Value
Interface	Conversational chat panel with message history
Model	MNIME-Core-V5-Q4_K_M.gguf (Qwen2.5 1.5B fine-tuned)
RAG	Queries first run through FAISS semantic search; top-5 chunks fed as context to the model
Context Window	4,096 tokens
Max Response	1,024 tokens
Prompt Format	ChatML (< im_start > / < im_end >)
Pop-out	Double-click to spawn magnetic translucent floating chat window

4.11 Reference Engine

Property	Value
Purpose	Highlight a section in one open document to auto-synthesize a comparative brief against all other open documents
Method	Selected text sent to NLPEngine.synthesize_reference() with semantic search results as context
Output	Comparative analysis: alignment, conflicts, and relationships across documents
System Role	Advanced legal and document analysis NLP persona

5 NLP ENGINE — MNIME-CORE

The MNIME NLP engine is a singleton (**NLPEngine**) that wraps **llama-cpp-python** to load and run the bundled GGUF model. It is designed for zero-configuration operation: the bundled model is auto-detected, GPU layers are fully auto-offloaded, and all memory is aggressively cleaned on toggle-off or application close.

Property	Detail
Base Model	Qwen2.5-1.5B-Instruct
Fine-tuning	Progressive multi-stage alignment: V3 doc synthesis + V4 philosophy + V5 anti-bias
Neural Engine	Neural Assimilation Engine (NAE): streaming fp16 TIES weight fusion
Quantization	Q4_K_M (4-bit, K-Quant Mixed) with high-fidelity Q8_0 and fp16 archives
Format	GGUF (llama.cpp compatible)
Parameters	~1.5 Billion
Context Length	4,096 tokens (n_ctx)
CPU Threads	8 (n_threads)
GPU Layers	-1 (auto-offload all layers that fit in VRAM)
Flash Attention	Enabled if supported by build; falls back gracefully
Memory Locking	Disabled (use_mlock=False) for broad compatibility
Singleton Pattern	One global instance (NLPEngine.get_instance()), thread-locked
Cleanup	atexit, SIGINT, SIGTERM, and crash handler all call unload_model()
Smart Bookmark	max_tokens=25, stop on newline — fast constrained title generation
Smart Filename	max_tokens=20, thread-locked for safety
Chat Response	max_tokens=1,024, stop on < im_end >
Reference Brief	max_tokens=1,024, structured comparative analysis prompt
HuggingFace	KyleDeanAI/MNIME-Core-V5-Q4_K_M

Loading & Lifecycle

The model is **not loaded at startup**. Click the **Reload NLP** button (refresh icon in the Tabs Bar) to load it into memory. This keeps startup instant. Uncheck the **NLP checkbox** to immediately unload the model and free all VRAM/RAM. The Reload NLP button glows cyan while the model is unloaded as a visual reminder.

6 SEMANTIC SEARCH ENGINE

The **SearchEngine** class provides FAISS-backed vector search over all queued documents. It supports PDFs (with Windows OCR fallback for image-only pages), plain text, Markdown, JSON, CSV, and common source code formats.

Property	Detail
Vector Index	FAISS (CPU)
Embedding Model	BAAI/bge-small-en-v1.5 (via langchain-huggingface)
Chunking	RecursiveCharacterTextSplitter — chunk_size=1500, overlap=200
Smart Sampling	Extracts top-20 probe terms from a 10% content sample; indexes only chunks containing those terms
Fallback	If smart sampling yields no chunks, falls back to full indexing
PDF Text	PyMuPDF page.get_text() per page
OCR	Windows OCR API (winsdk) on image-only PDF pages (2x pixmap)
Search	vectorstore.similarity_search(query, k=5) — returns top-5 chunks with source
Supported Formats	PDF, TXT, MD, JSON, CSV, PY, JS, TS, HTML, CSS, CPP, C, H, Java
Index Lifetime	Built on demand for NLP mode; cleared from memory with model unload

7 USER INTERFACE GUIDE

7.1 Tabs Bar

The Tabs Bar runs horizontally across the top of the main window. It is divided into three groups: **Left** (Edit, Bookmark, Reference), **Center** (JPG→PDF, TXT→PDF, Combine, Split, Compress, PDF→JPG, PDF→DOCX), and **Right** (NLP checkbox, NLP chat, Reload NLP).

Icon Mode	Tab Label	Description
EDIT	Edit	Open image or PDF in the Edit Suite (crop/rotate)
BOOKMARK	Bookmark	Add smart or heuristic bookmarks to a PDF
REFERENCE	Reference	Cross-reference documents with NLP synthesis
JPG → PDF	JPG → PDF	Convert image files to a unified PDF
TXT → PDF	TXT → PDF	Convert plain text files to native vector PDF
MERGE	Combine PDF	Merge multiple PDFs and images into one document
SPLIT	Split PDF	Split a PDF into individual page files
COMPRESS	Compress	Reduce PDF file size
PDF → JPG	PDF → JPG	Export PDF pages as high-res JPG images
PDF → DOCX	PDF → DOCX	Convert PDF to editable Word document
NLP (checkbox)	NLP	Master toggle — uncheck to unload model instantly
NLP (button)	NLP Chat	Open the conversational NLP interface
RELOAD NLP	Reload NLP	Load / reload the GGUF model into memory. Glows cyan when model is unloaded.
STATS	Stats	Open Stats telemetry dashboard with live dynamic line charts.

7.2 Drop Zone

The right side of the main window features a **Drop Zone** centered on the MNIME Penteract logo. Drag any supported file type directly onto the logo to queue it. A subtle drag-over highlight provides real-time feedback. Alternatively, click the neon-outlined **ADD FILES** button to open the custom file browser dialog.

7.3 Gallery Carousel

Once files are queued, they populate the **Gallery Carousel** on the left side as interactive file cards. Each card displays a thumbnail (rendered by PyMuPDF for PDFs, Pillow for images), filename, and status badge. Cards can be scrolled horizontally and dragged left/right to reorder — implemented via O(1) index-based layout shifts rather than full widget rebuilds. A neon-red **X** button in the top bar clears the entire queue.

7.4 Action Bar

The **Action Bar** at the bottom of the window contains the primary action button (label changes with mode, e.g. *MERGE FILES*, *COMPRESS PDF*) and a progress bar that appears during processing. All operations run in a **QThread** worker, so the UI remains fully interactive and animated at 60 FPS throughout.

7.5 Document Reader

MNIME includes a built-in frameless **Document Reader** that opens as an independent, resizable dark metallic window. Documents are rendered at **4.0x Retina pixel density** (288 DPI equivalent) for ultra-sharp, anti-aliased vector text. Use **Ctrl + Mouse Scroll** to zoom smoothly. The reader auto-fits the document to the window width with margin padding on open, and supports native smooth diagonal resizing.

7.6 Edit UI

The Edit UI (accessible via the **EDIT** tab) opens a dedicated editor window. For images, it provides crop and rotation controls. For PDFs, it allows cropping and rotating individual pages or all pages simultaneously. The editor renders at 4.0x Retina density for

precision alignment. Changes are saved back to the source file or a new output path.

7.7 NLP Floating Chat

Double-clicking the NLP console panel spawns a **magnetic, translucent floating chat window** that mirrors the main window's dark metallic aesthetic. The window is perfectly synced with the main application state and stays on top for convenient side-by-side document interaction. It can be repositioned freely and snaps back into the main window on double-click.

8 INSTALLATION & SETUP

8.1 Standalone Installer (Recommended)

Two pre-built installers are provided in the `installer/` folder:

Installer	Description	Install Location
MNIME_installer.exe	Premium animated installer with custom PyQt6 UI — branded dark window, animated flying-file progress bar, and automatic shortcut creation.	%LOCALAPPDATA%\Programs\MNIME

8.2 Run from Source

Step 1. Download the NLP Model: Go to huggingface.co/KyleDeanAI/MNIME-Core-V5-Q4_K_M, download the `.gguf` file, and place it in the `models/` directory.

Step 2. Run `setup.bat`: Creates a Python 3.12+ virtual environment and installs all dependencies from `requirements.txt`.

```
setup.bat
```

Step 3. Launch: Double-click `run.bat`, or from terminal:

```
.venv\Scripts\python.exe MNIME.py
```

Step 4. Pin to Taskbar: Run `create_shortcut.bat` to generate a Desktop shortcut, then right-click → Pin to taskbar.

8.3 Build Pipeline

Run `build_app.bat`. Ensure the `.gguf` model is in `models/` first — it is bundled into the installer.

Step	Action
1	Create/update <code>.venv</code> and install all build dependencies
2	Compile main app with PyInstaller using <code>MNIME.spec</code> → <code>dist/MNIME/</code>
3	Build animated installer <code>installer/MNIME_installer.exe</code> via PyInstaller + <code>custom_installer.py</code>

9 KEYBOARD SHORTCUTS & TIPS

Shortcut / Action	Effect
Ctrl + Mouse Scroll	Zoom in/out in Document Reader and Edit UIs
Drag file onto logo	Queue file for processing
Drag card left/right	Reorder files in the Gallery Carousel
Double-click NLP panel	Spawn magnetic translucent floating chat window
Click NLP checkbox	Toggle NLP master switch (loads/unloads model)
Click Reload NLP button	Load or reload the GGUF model into memory
Click X on card	Remove that file from the queue
Click red X (top bar)	Clear entire file queue
Click ADD FILES	Open custom dark-mode file browser dialog
Click primary action button	Execute the current mode's operation

Tips

- Keep the **NLP checkbox unchecked** for ultra-lightweight operation when you only need PDF tools.
- For **Split PDF with Smart Naming**, load the NLP model first — the model generates meaningful filenames per page.
- For large file batches, MNIME runs document operations on a background worker so the UI stays responsive.
- The **Compress PDF** tool is most effective on PDFs with many embedded images or unoptimized streams.
- The **Reference Engine** works best when multiple documents are already queued in the carousel.
- Ctrl+Scroll zoom in the reader is continuous and smooth — there is no discrete zoom step limit.

10 PERFORMANCE NOTES

Optimization	Technique
C-Accelerated PDF Engine	All core PDF operations use PyMuPDF's native C-level routines — up to 50x faster than pure-Python alternatives with negligible RAM footprint
O(1) Carousel Operations	Drag-and-drop card reorder and removal use surgical layout index shifts; widget tree never torn down even with 5,000 cards loaded
Dynamic Memory Management	GGUF model and FAISS vector index completely cleared from memory on NLP toggle-off or app close — no background memory hoarding
Manual NLP Control	Model stays unloaded at startup; user explicitly loads it, keeping startup time instant
In-Memory Pixmap Caching	Card thumbnails and SVG icons rasterized and pre-scaled once; no CPU resampling during scroll or hover events
Hardware-Accelerated Cards	Heavy drop shadow textures replaced with pure stylesheet hardware borders — UI scrolls smoothly with 5,000 files
60 FPS Non-Blocking UI	All processing (PDF ops, NLP inference, indexing) runs in QThread workers; main thread never blocked
Image Conversion	JPG→PDF and PDF→JPG run sequentially on a background worker (MuPDF is not thread-safe)
Split PDF	Pages processed sequentially; filenames sanitized for Windows safety
Flash Attention	Enabled by default for llama-cpp-python if build supports it; graceful fallback if not
GPU Auto-Offload	n_gpu_layers=-1 tells llama.cpp to offload as many layers as will fit in VRAM automatically

11 DEPENDENCY STACK

Package	Version	Role
PyQt6	6.11.0	Desktop UI framework
pymupdf	1.28.2	Primary PDF/image engine (C-level)
pypdf	6.19.0	PDF fallback engine
pdf2docx	0.5.13	PDF → DOCX conversion
python-docx	1.2.0	DOCX text fallback
Pillow	12.3.0	Image processing fallback
llama-cpp-python	0.3.35	GGUF model inference
faiss-cpu	1.15.1	Vector similarity index
langchain	0.3.30	RAG pipeline orchestration
langchain-community	0.3.21	FAISS vectorstore integration
langchain-huggingface	0.3.1	Embedding model loading
sentence-transformers	6.1.0	BAAI/bge-small-en-v1.5 embeddings
torch	2.14.1	Transformer backend
transformers	5.18.0	HuggingFace model utilities
numpy	2.5.3	Numerical operations (≥2.0 required)
opencv-python-headless	4.11.0.86	Image processing utilities
winsdk	latest	Windows OCR API for image PDFs
scikit-learn	1.9.1	ML utilities for search
pandas	2.3.3	Data frame operations in search
pyinstaller	6.22.3	Application packaging

12 ERROR HANDLING & VALIDATION

MNIME implements defensive error handling at every layer to prevent crashes and provide actionable feedback without interrupting the user's workflow.

Scenario	Behavior
Wrong file type for selected mode	Intelligent intercept before processing; prompt shown without crash
Empty file queue on action	Validated before action; helpful prompt displayed
NLP model missing at startup	Engine initializes silently; error surfaced only when user clicks Reload NLP
NLP disabled via checkbox	Model immediately unloaded; subsequent NLP calls return informative error string
PyMuPDF failure on any operation	Automatic fallback to pypdf + Pillow for merge/compress/convert
pdf2docx import failure	Fallback to pypdf + python-docx text extraction
OCR engine unavailable	Returns empty string; page indexed without text
GPU layer offload exceeds VRAM	llama.cpp automatically reduces layers to CPU
Flash Attention unsupported	TypeError caught; retried without flash_attn kwarg
NumPy <2.0 on Python 3.13+	Requirement pinned to numpy>=2.0.0 to prevent longdouble overflow crash
FAISS index empty after smart sampling	Automatic fallback to full indexing
Per-page exception in split/convert	Per-page try/except; failed pages logged, others continue
Application crash	sys.excepthook overridden to call unload_model() before propagating
Process termination (SIGINT/SIGTERM)	Signal handlers call unload_model() then sys.exit(0)

13 CHANGELOG

MNIME Core V5 — Multi-Stage Alignment & Neural Assimilation

- Added: Multi-stage progressive alignment pipeline combining V3 document synthesis, V4 cloud philosophy & adversarial robustness, and V5 sociological anti-bias alignment.
- Added: Neural Assimilation Engine (NAE v2.0) featuring streaming fp16/bf16 safetensors TIES-merging, gradient delta isolation, and sign consensus resolution.
- Added: Multi-precision model artifacts including full-precision safetensors, 16-bit GGUF (3.09 GB), 8-bit Q8_0 GGUF (1.64 GB), and quantized Q4_K_M GGUF (~1.0 GB).
- Improved: Ethical reasoning engine actively deconstructs hate tropes and demographic stereotypes using empirical sociology rather than generic refusal templates.

MNIME Final — Bundled NLP Model

- Changed: The fine-tuned MNIME-Core-V5-Q4_K_M.gguf model is now bundled directly inside the application under models/. No external model download required.
- Removed: The Settings gear icon and NLP hardware configuration dialog have been removed. Hardware offloading is handled automatically at runtime.
- Removed: The finetuning workflow (training/) is no longer part of the repository. The model is shipped as a finished artifact.
- Added: Custom animated PyQt6 installer (MNIME_installer.exe).

MNIME — UI & UX Complete Overhaul

- Added: Procedurally generated 5D Penteract branding logo with true mathematical 3D depth-sorting and an independent orbiting neon file.
- Added: Advanced High-Resolution 4.0x Retina rendering pipeline for the PDF Reader and Edit UIs.
- Added: Fluid Ctrl+Scroll mouse wheel zoom capabilities across all document viewer and editor viewports.
- Improved: The Reader UI has been completely decoupled from the main window, featuring its own independent resizable frameless dark metallic window.
- Improved: The Image and PDF Edit UIs have been fully upgraded to the MNIME translucent dark metallic theme.

Version 2.1 — Compatibility & Stability

- Fixed: Model loading crash on Python 3.13+ caused by a longdouble overflow in NumPy 1.x getlimits.py.
- Updated: NumPy dependency bumped to >=2.0.0. NumPy 2.x resolves the broken _register_known_types initialization on Windows with Python 3.13+.
- Updated: pyproject.toml now correctly lists numpy>=2.0.0 and llama-cpp-python>=0.2.75 as explicit dependencies.

Version 2.0 — Initial Public Release

- Full feature set: Merge, Edit, JPG↔PDF, TXT→PDF, Split, Compress, DOCX export, Semantic Bookmarks, NLP/RAG chat, Cross-Reference engine.
- Free-floating dark metallic PyQt6 UI with physics particle transitions.
- Local offline GGUF model integration via llama-cpp-python.
- Custom dark-mode file browser dialog replacing OS file picker.
- FAISS-backed semantic search with BAAI/bge-small-en-v1.5 embeddings.

14 LICENSE & ATTRIBUTION

Open Source License

MNIME is distributed as open source software. See the **LICENSE** file in the repository root for the full license text.

Component	Author / Source
MNIME Application	kyledeanml / KyleDeanAI
MNIME-Core NLP Model	KyleDeanAI — fine-tuned from Qwen2.5-1.5B-Instruct
Base Model (Qwen2.5-1.5B)	Alibaba Cloud — Qwen Team
PyMuPDF	Artifex Software
llama-cpp-python	Andrei Betlen
FAISS	Meta AI Research
LangChain	LangChain, Inc.
BAAI/bge-small-en-v1.5	Beijing Academy of Artificial Intelligence
PyQt6	Riverbank Computing / The Qt Company
pdf2docx	Artifex / dothinking

MNIME — Multimodal Neural Interface Machine Extension
Private & Offline — Zero External Uploads — Python 3.12+ / PyQt6 / Windows 10+
<https://github.com/kyledeanml/MNIME>